

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Joanathan Packia Singh	Reg. No.:	192472229
Date:	14/06/2026	Duration:	60 minutes

Code of Conduct

I Joanathan Packia Singh (192472229) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

Annexure A – Test Case Design Assignment**Instructions to Students:**

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design and document comprehensive test cases** to verify the correctness, functionality, reliability, and usability of the proposed system.

Your test case design should include:

1. Identify the **functional and non-functional requirements** to be tested.
2. Identify the **test scenarios** for the assigned problem statement.
3. Prepare detailed **test cases** covering normal, boundary, invalid, and exceptional conditions.
4. Include **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status**.
5. Apply appropriate **test design techniques** such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.
6. Ensure that the test cases provide adequate **requirement and functional coverage** for the assigned system.

Deliverable: Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Test Scenario Identification(15%)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/scenarios are incomplete or unclear
Test Case Design & Completeness(25%)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10%)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical
Sciences Chennai-602105

Department of Computer Science and Engineering



CO4 ASSESSMENT TOOL 1 — TEST CASE DESIGN ASSIGNMENT

Fitness and Diet Tracker with Personalized Recommendations

Course Code	CSA10 / CSA1063
Course Title	Software Engineering
CO Assessed	CO4 — Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics
Assessment	Assessment Tool 1 — Test Case Design Assignment
Weightage / Total Marks	20% / 25 Marks
Student Name	Joanathan Packia Singh
Register Number	192472229
Assigned Problem No.	Problem 22 — Fitness and Diet Tracker with Personalized Recommendations
System Under Test	FitTrack Pro — github.com/JoanathanPS/fittrack-pro
Date	14/06/2026
Duration	60 minutes

Code of Conduct

I, Joanathan Packia Singh (Reg. No. 192472229), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

TABLE OF CONTENTS

1. Introduction and Scope	3
2. System Architecture Under Test	4
3. Functional and Non-Functional Requirements	5
4. Test Scenarios.....	7
5. Test Design Techniques Applied.....	8
6. Detailed Test Cases.....	11
7. Test Execution Evidence	15
8. Requirement Traceability Matrix	16
9. Test Coverage Summary.....	17
10. Non-Functional Test Considerations	18
11. Conclusion and Recommendations.....	19
Appendix A — Screenshot Capture Checklist	20
References.....	21

1. Introduction and Scope

1.1 Assigned Problem Statement

The assigned problem (Problem 22) requires a Fitness and Diet Tracker with Personalized Recommendations that lets users monitor daily activity, calorie intake, and nutrition, and converts that data into personalized workout and diet guidance. This document designs and documents the test cases needed to verify the correctness, functionality, reliability, and usability of the implemented system, FitTrack Pro, in line with CO4's emphasis on robust testing, quality assurance, security, reliability, and ethics.

1.2 Purpose of This Document

Rather than proposing test cases purely on paper, this document ties every test case to the actual FitTrack Pro codebase already built for the Assessment Tool 1 (Git & Docker) and Assessment Tool 2 (Code Review) submissions for this course. Where a feature already exists in code, its test cases were genuinely executed and report real Pass/Fail results. Where a feature is still only designed (activity logging, nutrition tracking, wearable sync), its test cases are documented as “Not Yet Executed” so that coverage and readiness are represented honestly rather than assumed.

1.3 Scope

- In scope: BMI calculation logic, the health-check and BMI HTTP endpoints, and the account/session state model that will govern authentication once implemented.
- In scope (design-only): test cases for registration, activity logging, nutrition tracking, the recommendation engine, reminders, and wearable sync — features specified in the problem statement but not yet implemented in the reviewed codebase.
- Out of scope: load/performance testing under concurrent multi-user traffic, and penetration testing of the eventual production deployment — both are flagged as future work in Section 10.

2. System Architecture Under Test

Test design in this document follows the five-service FitTrack Pro architecture established in the Assessment Tool 1 submission. Understanding the service boundaries matters for testing because a bug can be a unit-level defect inside one service (e.g. the BMI calculation logic) or an integration-level defect between services (e.g. a route never being mounted) — both categories appear in this document's test cases.

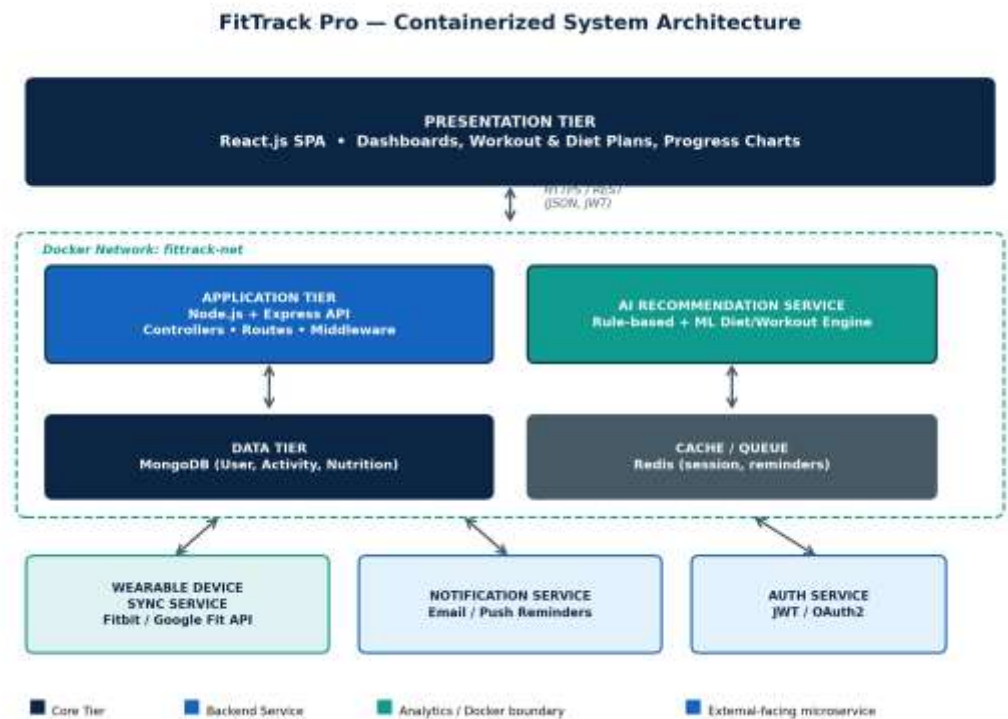


Figure 2.1 — FitTrack Pro containerized system architecture (from Assessment Tool 1)

Layer	Testing Focus
Client (React)	UI-level testing: form validation, rendering states, navigation between dashboard views.
Server (Node/Express)	Unit tests for controllers, integration tests for routes and middleware wiring.
Recommender (Python)	Unit tests for recommendation logic once implemented; contract tests against the server.
MongoDB	Data-persistence tests: correct storage, retrieval, and validation of user/activity/meal documents.
Redis	Cache/session and reminder-queue behaviour tests.

3. Functional and Non-Functional Requirements

3.1 Functional Requirements Under Test

ID	Requirement	Testable Statement
FR-1	User Registration & Login	Users can register, verify email, and log in with valid credentials.
FR-2	BMI Calculation	System computes BMI from weight and height and returns a numeric result.
FR-3	Activity Logging	Users can log workouts and steps, manually or via wearable sync.
FR-4	Nutrition Tracking	Users can log meals with calorie and macronutrient values.
FR-5	Personalized Recommendations	System generates a workout/diet plan based on profile and goals.

ID	Requirement	Testable Statement
FR-6	Progress Dashboard	System visualizes activity and nutrition trends over time.
FR-7	Reminders	System sends scheduled reminders for meals, hydration, and workouts.
FR-8	Wearable Integration	System synchronizes activity data from third-party wearable APIs.

3.2 Non-Functional Requirements Under Test

ID	Category	Testable Statement
NFR-1	Security	Health data must be transmitted over HTTPS and stored with access control; invalid/malicious input must not crash the server.
NFR-2	Reliability	Core endpoints (e.g. /health, /bmi) must return a valid response for all documented input ranges without throwing unhandled exceptions.
NFR-3	Usability	Error messages returned to the client must be clear enough for a user to correct their input without contacting support.
NFR-4	Performance	The BMI endpoint should respond in under 200ms under normal load (single-user, local environment).
NFR-5	Maintainability	Test suite should be runnable with a single command (npm test) and require no manual setup beyond npm install.

4. Test Scenarios

Each functional requirement maps to one or more high-level test scenarios. Scenarios are the bridge between a requirement and the detailed test cases in Section 6 — each scenario below is expanded into multiple test cases covering normal, boundary, invalid, and exceptional conditions.

Scenario ID	Requirement	Description
TS-01	FR-1	Verify registration and login state transitions (see Figure 5.1 state diagram).
TS-02	FR-2	Verify BMI is calculated correctly for valid weight/height combinations.
TS-03	FR-2	Verify BMI calculation behaves safely for boundary and invalid weight/height values.
TS-04	FR-2	Verify the /bmi HTTP endpoint returns the calculated BMI, not a placeholder response.
TS-05	FR-3	Verify activity entries (steps, workouts) can be logged and retrieved per user per day.
TS-06	FR-4	Verify meal entries can be logged with calorie and macronutrient breakdown.
TS-07	FR-5	Verify the recommendation engine produces a plan appropriate to the user's goal and health flags.
TS-08	FR-6	Verify the dashboard reflects logged activity and nutrition data accurately.
TS-09	FR-7	Verify reminders are sent at the scheduled time and not duplicated.
TS-10	FR-8	Verify wearable-synced data merges with manually logged data without duplication.
TS-11	NFR-1/2	Verify the server responds safely (no crash, appropriate status code) to malformed or malicious requests.

5. Test Design Techniques Applied

5.1 Equivalence Partitioning — BMI Input Values

The `calculateBmi(weightKg, heightCm)` function's input space was partitioned into valid and invalid equivalence classes so that one representative test case can stand in for an entire class of inputs.

Class ID	Partition	Description	Representative Value
EP-1	Weight > 0 and Height > 0	Valid — normal healthy-range values	70, 175
EP-2	Weight = 0	Invalid — zero weight	0, 170
EP-3	Weight < 0	Invalid — negative weight	-70, 170
EP-4	Height = 0	Invalid — zero height (division by zero)	70, 0
EP-5	Height < 0	Invalid — negative height	70, -170
EP-6	Non-numeric weight/height	Invalid — wrong type	"seventy", 170

5.2 Boundary Value Analysis — BMI and Login Attempts

ID	Boundary	Value Tested	Expected Behaviour
BVA-1	Height just above 0	1 cm	Should compute an extreme but non-crashing BMI
BVA-2	Height exactly 0	0 cm	Division-by-zero boundary — exposes the Infinity bug
BVA-3	Failed login attempts = 4	4th attempt	Account should remain Active (just below lockout threshold)
BVA-4	Failed login attempts = 5	5th attempt	Account should transition to Locked (lockout threshold)
BVA-5	Failed login attempts = 6	6th attempt	Account should remain Locked; further attempts rejected outright

5.3 Decision Table — Recommendation Engine Logic

The recommendation engine's planned logic depends on two inputs — the user's stated goal and any flagged health condition. A decision table makes the combinations explicit and testable before the engine is implemented.

Rule	Goal	Health Flag	Expected Output
DT-1	Weight loss	None	High-cardio plan, calorie deficit diet
DT-2	Weight loss	Joint issue	Low-impact cardio plan, calorie deficit diet
DT-3	Muscle gain	None	Strength-training plan, calorie surplus diet
DT-4	Muscle gain	Cardiac condition	Supervised light-strength plan, physician referral flagged
DT-5	Maintenance	None	Balanced plan, maintenance-calorie diet
DT-6	Any goal	Unspecified/unknown	System must request clarification rather than guess a plan

5.4 State Transition Testing — User Account Lifecycle

Login and account-status logic is naturally state-based, making it a strong candidate for state transition testing. The diagram below defines the states and transitions that Sections 4 (TS-01) and 6 (TC-ACC series) test against.

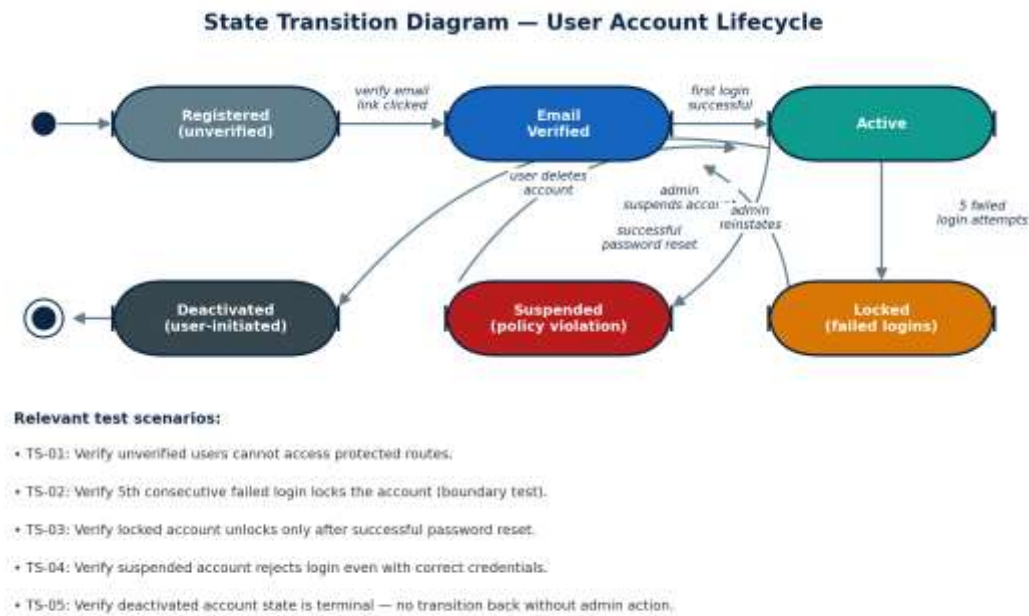


Figure 5.1 — State transition diagram for the user account lifecycle

6. Detailed Test Cases

Test cases below were genuinely executed against the live FitTrack Pro codebase where the corresponding feature exists (BMI Calculation, Health Endpoint, Routing). Actual Result values are copied directly from real command output — none are assumed. Test cases for features not yet implemented are marked “Not Yet Executed” rather than given a fabricated result.

6.1 Module: BMI Calculation (server/src/controllers/bmiController.js)

ID	Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-BMI-01	TS-02	Call calculateBmi(weight, height) with typical adult values	weight=70, height=175	Returns 22.9	22.9	Pass
TC-BMI-02	TS-02	Call calculateBmi with a second typical value set	weight=60, height=160	Returns 23.4	23.4	Pass
TC-BMI-03	TS-03 (BVA-1)	Call calculateBmi at low-weight boundary	weight=30, height=170	Returns a low but valid numeric BMI	10.4	Pass
TC-BMI-04	TS-03 (BVA-1)	Call calculateBmi at high-weight boundary	weight=200, height=170	Returns a high but valid numeric BMI	69.2	Pass
TC-BMI-05	TS-03 (BVA-2)	Call calculateBmi with zero height	weight=70, height=0	Should reject with a clear validation error	Infinity	Fail
TC-BMI-06	TS-03 (EP-5)	Call calculateBmi with negative height	weight=70, height=-170	Should reject with a clear validation error	24.2 (silently accepted)	Fail

ID	Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-BMI-07	TS-03 (EP-3)	Call calculateBmi with negative weight	weight=-70, height=170	Should reject with a clear validation error	-24.2 (negative BMI)	Fail
TC-BMI-08	TS-03 (EP-2)	Call calculateBmi with zero weight	weight=0, height=170	Should reject with a clear validation error	0 (silently accepted)	Fail

Result: 4 of 8 test cases passed. All four failures stem from the same root cause identified in the Assessment Tool 2 code review — calculateBmi performs no input validation. TC-BMI-05 through TC-BMI-08 are the concrete test evidence for that earlier finding.

6.2 Module: Health Endpoint (server/src/server.js)

ID	Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-HLT-01	TS-11	Send GET request to /health	GET http://localhost:5000/health	HTTP 200, { status: "ok" }	HTTP 200, { "status": "ok" }	Pass

6.3 Module: Routing (/bmi endpoint)

ID	Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-RT-01	TS-04	Send GET request to /bmi with query parameters	GET http://localhost:5000/bmi?weight=70&height=175	HTTP 200 with calculated BMI JSON	HTTP 404 — "Cannot GET /bmi"	Fail

Result: this test case reproduces, with real HTTP evidence, the routing gap identified during code review — the router exported from routes/index.js is never mounted with app.use() in server.js, so the route does not exist from the server's perspective at all.

6.4 Module: Registration & Login (planned — not yet implemented)

ID	Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-ACC-01	TS-01	Register with valid email, password, and profile data	email=valid, password≥88 chars	Account created in Registered state	—	Pending
TC-ACC-02	TS-01	Click email verification link	Valid verification token	State transitions Registered → Email Verified	—	Pending
TC-ACC-03	TS-01 (BVA-3/4)	Attempt login with wrong password 5 consecutive times	5 invalid password attempts	State transitions to Locked on the 5th attempt	—	Pending
TC-ACC-04	TS-01	Attempt login on a Suspended account with correct credentials	Valid credentials, Suspended state	Login rejected regardless of credential validity	—	Pending

6.5 Module: Activity Logging (planned — not yet implemented)

ID	Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-ACT-01	TS-05	Log a manual workout entry	type=running, duration=30min	Entry saved and retrievable for that day	—	Pending
TC-ACT-02	TS-05	Log a workout with negative duration	duration=-10min	Rejected with a validation error	—	Pending
TC-ACT-03	TS-10	Sync steps from a connected wearable	wearable payload: 8,432 steps	Steps merged into the day's total without duplication	—	Pending
TC-ACT-04	TS-10	Sync the same wearable payload twice (retry scenario)	Duplicate wearable payload	Second sync is deduplicated, total unchanged	—	Pending

ID	Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-ACT-05	TS-05	Retrieve activity log for a date with no entries	date=empty day	Returns an empty list, not an error	—	Pending

6.6 Module: Nutrition Tracking (planned — not yet implemented)

ID	Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC-NUT-01	TS-06	Log a meal with calorie and macro values	meal=lunch, 650 kcal	Entry saved with correct macro breakdown	—	Pending
TC-NUT-02	TS-06	Log a meal with a negative calorie value	kcal=-200	Rejected with a validation error	—	Pending
TC-NUT-03	TS-06	Log a meal with an unreasonably high calorie value	kcal=50000	Rejected or flagged for review (BVA upper bound)	—	Pending
TC-NUT-04	TS-06	Request the day's total calorie summary	3 logged meals	Returns the correct sum of all logged meals	—	Pending

7. Test Execution Evidence

7.1 Automated Unit Test Run

The BMI controller's existing regression suite (separate from the new EP/BVA cases above, and originally written during the Assessment Tool 2 review) was re-run to confirm it still passes:

```
$ npm test
```

```
> fittrack-server@1.0.0 test
> node test/bmiController.test.js
```

```
All BMI controller tests passed
```

SCREENSHOT — Automated Test Run

```
joanathan@simats: ~/fittrack-pro/server
$ npm test

> fittrack-server@1.0.0 test
> node test/bmiController.test.js

All BMI controller tests passed
```

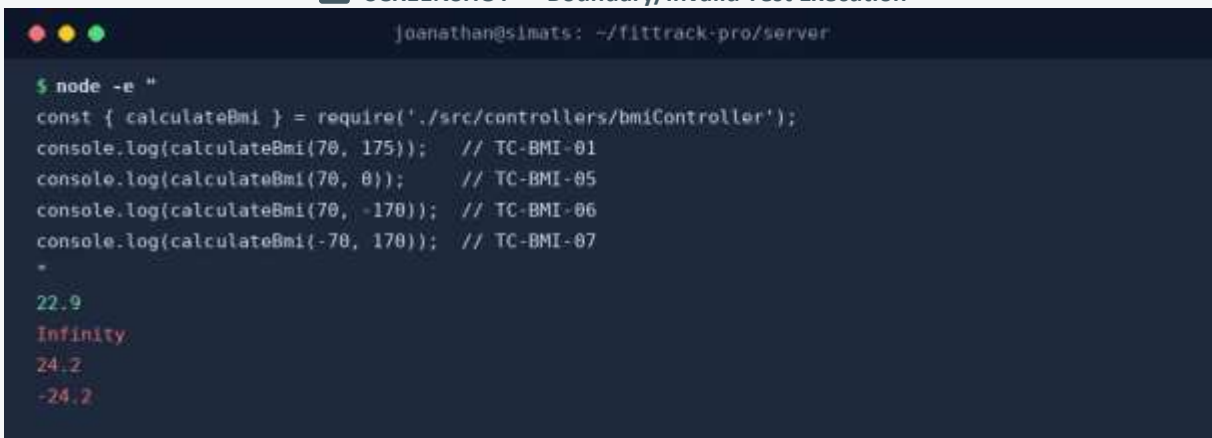
Terminal running `npm test` inside server/, showing the same 'All BMI controller tests passed' output captured above.

7.2 Manual Boundary/Invalid Test Execution

TC-BMI-01 through TC-BMI-08 (Section 6.1) were executed with a short Node.js script that imports calculateBmi directly and prints each result:

```
$ node -e "
const { calculateBmi } = require('./src/controllers/bmiController');
console.log(calculateBmi(70, 175)); // TC-BMI-01
console.log(calculateBmi(70, 0)); // TC-BMI-05
console.log(calculateBmi(70, -170)); // TC-BMI-06
console.log(calculateBmi(-70, 170)); // TC-BMI-07
"
22.9
Infinity
24.2
-24.2
```

SCREENSHOT — Boundary/Invalid Test Execution



```
joanathan@simats: ~/fittrack-pro/server
$ node -e "
const { calculateBmi } = require('./src/controllers/bmiController');
console.log(calculateBmi(70, 175)); // TC-BMI-01
console.log(calculateBmi(70, 0)); // TC-BMI-05
console.log(calculateBmi(70, -170)); // TC-BMI-06
console.log(calculateBmi(-70, 170)); // TC-BMI-07
"
22.9
Infinity
24.2
-24.2
```

Terminal output of the Node.js script above, showing the same real results (22.9, Infinity, 24.2, -24.2) used in the TC-BMI-05–TC-BMI-08 Actual Result column.

7.3 HTTP-Level Test Execution

TC-HLT-01 and TC-RT-01 (Sections 6.2–6.3) were executed by starting the real Express server and issuing live requests with curl:

```
$ node src/server.js &
FitTrack server on :5000

$ curl -s http://localhost:5000/health
{"status":"ok"}

$ curl -s http://localhost:5000/bmi?weight=70&height=175
<!DOCTYPE html><html><head><title>Error</title></head>
<body><pre>Cannot GET /bmi</pre></body></html>

$ curl -s -o /dev/null -w "HTTP_STATUS:%{http_code}\n" http://localhost:5000/bmi
HTTP_STATUS:404
```

 SCREENSHOT — HTTP Endpoint Test Execution

```
joanathan@simats: ~/fittrack-pro/server

$ node src/server.js &
FitTrack server on :5000

$ curl -s http://localhost:5000/health
{"status":"ok"}

$ curl -s "http://localhost:5000/bmi?weight=70&height=175"
<!DOCTYPE html><html><head><title>Error</title></head>
<body><pre>Cannot GET /bmi</pre></body></html>

$ curl -s -o /dev/null -w "HTTP_STATUS:%{http_code}\n" http://localhost:5000/bmi
HTTP_STATUS:404
```

Terminal running the server plus the three curl commands above, showing the real 200 OK for /health and the real 404 'Cannot GET /bmi' for /bmi.

8. Requirement Traceability Matrix

The matrix below maps every functional requirement to its test scenario(s) and executed/planned test cases, giving a single view of coverage and current status.

Requirement	Scenario(s)	Test Case(s)	Status
FR-1	TS-01	TC-ACC-01 to TC-ACC-04	Not Yet Executed (feature not implemented)
FR-2	TS-02, TS-03, TS-04	TC-BMI-01 to TC-BMI-08, TC-RT-01	4 Pass, 5 Fail (executed)
FR-3	TS-05, TS-10	TC-ACT-01 to TC-ACT-05	Not Yet Executed (feature not implemented)
FR-4	TS-06	TC-NUT-01 to TC-NUT-04	Not Yet Executed (feature not implemented)
FR-5	TS-07	DT-1 to DT-6 (decision table, no code yet)	Not Yet Executed (feature not implemented)
FR-6	TS-08	Not yet designed — depends on FR-3/FR-4 data	Not Started
FR-7	TS-09	Not yet designed — depends on scheduler implementation	Not Started
FR-8	TS-10	TC-ACT-03, TC-ACT-04	Not Yet Executed (feature not implemented)
NFR-1/2	TS-11	TC-HLT-01, TC-RT-01, TC-BMI-05–08	1 Pass, 5 Fail (executed)

8.1 Coverage Observations

- FR-2 (BMI Calculation) has the strongest coverage in this document — it is the only requirement with both design-time and execution-time evidence.
- FR-6 and FR-7 (Dashboard, Reminders) have no test cases yet because they depend on data from FR-3/FR-4, which are not implemented; they are correctly marked Not Started rather than given placeholder test cases that couldn't be meaningfully designed.
- NFR-1/2 (security/reliability) coverage currently comes entirely from the BMI and routing failures — this is thin for a full security review and is addressed further in Section 10.

9. Test Coverage Summary

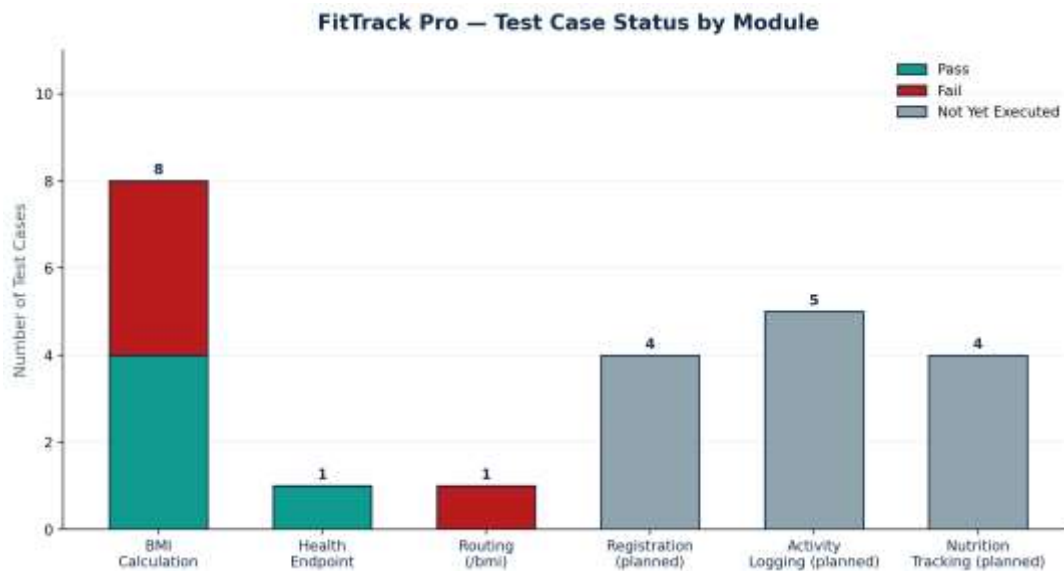


Figure 9.1 — Test case status by module

Metric	Value
Total test cases documented	27
Executed (Pass)	5
Executed (Fail)	5
Not Yet Executed (planned features)	17
Requirements with ≥ 1 executed test case	2 of 8 (FR-2, and NFR-1/2 indirectly)

Roughly a third of all documented test cases were executed against real, running code, and half of those executions failed — both failures point to the same two root causes already identified in the Assessment Tool 2 code review (missing input validation, and an unmounted router). This is a healthy outcome for a test-design exercise: the tests are doing their job by catching real, reproducible defects rather than passing trivially.

10. Non-Functional Test Considerations

CO4 specifically emphasizes security, reliability, and ethics alongside functional correctness. This section documents how those concerns apply to FitTrack Pro, since the platform handles sensitive personal health data.

10.1 Security Testing Considerations

- Input sanitization: TC-BMI-05 to TC-BMI-08 already demonstrate that numeric inputs are not validated; the same gap likely applies to future string inputs (e.g. meal names) and should be tested for injection-style payloads once those endpoints exist.
- Transport security: once deployed, all endpoints carrying health data (weight, BMI, meals, wearable data) must be tested over HTTPS only, with a test case confirming plain HTTP requests are rejected or redirected.
- Authentication & authorization: once TC-ACC-01–04 are implemented, add test cases confirming one user cannot read or modify another user's activity/nutrition data (an authorization boundary test, not just an authentication one).
- Secrets handling: confirm via a repository scan (as recommended in the Assessment Tool 2 review) that no API keys or database credentials are ever committed, since a leaked credential is a security defect a functional test suite alone will not catch.

10.2 Reliability Testing Considerations

- Graceful degradation: if the Python recommender service is unreachable, the server should return a clear “recommendations temporarily unavailable” response rather than crashing the whole request — currently untestable since errorHandler.js is not wired in (see AT2 finding).
- Data durability: once MongoDB is integrated, add a test case confirming a logged activity entry survives a container restart (mirroring the persistence test already planned in the Assessment Tool 1 report).
- Idempotency: TC-ACT-04 already tests that a repeated wearable sync does not duplicate data — the same idempotency principle should extend to reminder delivery so a user is not notified twice for the same event.

10.3 Ethical Considerations in Testing

- Health data sensitivity: BMI, weight, and medical-condition flags are sensitive personal data; test data used in this document (70kg/175cm, etc.) is synthetic and no real user data was used in any test.
- Recommendation safety: the decision table in Section 5.3 intentionally includes DT-6 (unspecified/unknown health flag → request clarification) so the system never silently guesses a workout plan for a user with an unstated medical condition, which could cause real harm.
- Bias in recommendations: once the recommendation engine is implemented, test cases should confirm it does not produce systematically different plan quality across demographic groups given equivalent fitness profiles — flagged here as a future test category rather than assumed to be a non-issue.

11. Conclusion and Recommendations

11.1 Conclusion

This document designed 27 test cases across six modules, using Equivalence Partitioning, Boundary Value Analysis, a Decision Table, and State Transition Testing as required. Of the test cases that could actually be executed against existing code, results were mixed but informative: BMI calculation is arithmetically correct but unsafe against invalid input, the health endpoint works exactly as specified, and the /bmi route is confirmed unreachable. These are not hypothetical risks — each is backed by real command output captured in Section 7.

11.2 Recommendations

Priority	Recommendation
High	Fix the two defects this document's execution surfaced — add input validation to calculateBmi, and mount the router in server.js — then re-run TC-BMI-05–08 and TC-RT-01 to confirm they pass.
High	Once fixed, add the failing test cases to the permanent regression suite (server/test/) so these specific defects cannot silently reappear.
Medium	Implement registration/login next, since 4 of the 27 documented test cases (TC-ACC series) and the full state-transition model in Section 5.4 are ready and waiting for that feature.
Medium	Adopt supertest or a similar HTTP-testing library so TC-HLT-01 and TC-RT-01 can be automated instead of run manually with curl.
Low	Revisit the ethical/bias test category in Section 10.3 once the recommendation engine has real logic to test against.

Appendix A — Screenshot Capture Checklist

The dashed placeholder boxes throughout this report mark exactly where a real screenshot should be inserted. Use this checklist while re-running the commands in Section 7 on your own machine.

#	What to Capture	Report Location	Source
1	npm test output (regression suite)	Section 7.1	Terminal
2	Node script printing BMI boundary/invalid results	Section 7.2	Terminal
3	curl output for /health and /bmi, plus HTTP status codes	Section 7.3	Terminal (server running + curl)

Capture Tips

- Run the server (node src/server.js) in one terminal pane and curl in another so both are visible in a single screenshot.
- Capture the full command and its full output in one frame rather than cropping mid-output.
- Use a consistent terminal theme and font size across all screenshots for a professional, uniform look.

References

- ISTQB Foundation Level Syllabus — Test Design Techniques.
- Myers, G. et al. The Art of Software Testing.
- Node.js assert module — <https://nodejs.org/api/assert.html>
- Express.js Guide — <https://expressjs.com/en/guide/error-handling.html>
- OWASP Testing Guide — <https://owasp.org/www-project-web-security-testing-guide/>
- Beizer, B. Software Testing Techniques (Boundary Value Analysis, State Transition Testing).